Introduction to Computer Architecture and Systems

Chapter 1

Essentials of Computer Architecture, Qualitative and Quantitative

Fengkai Wang(Iori Oikawa), Fall 2025 @ Drew University

Previously of CSCI 260…

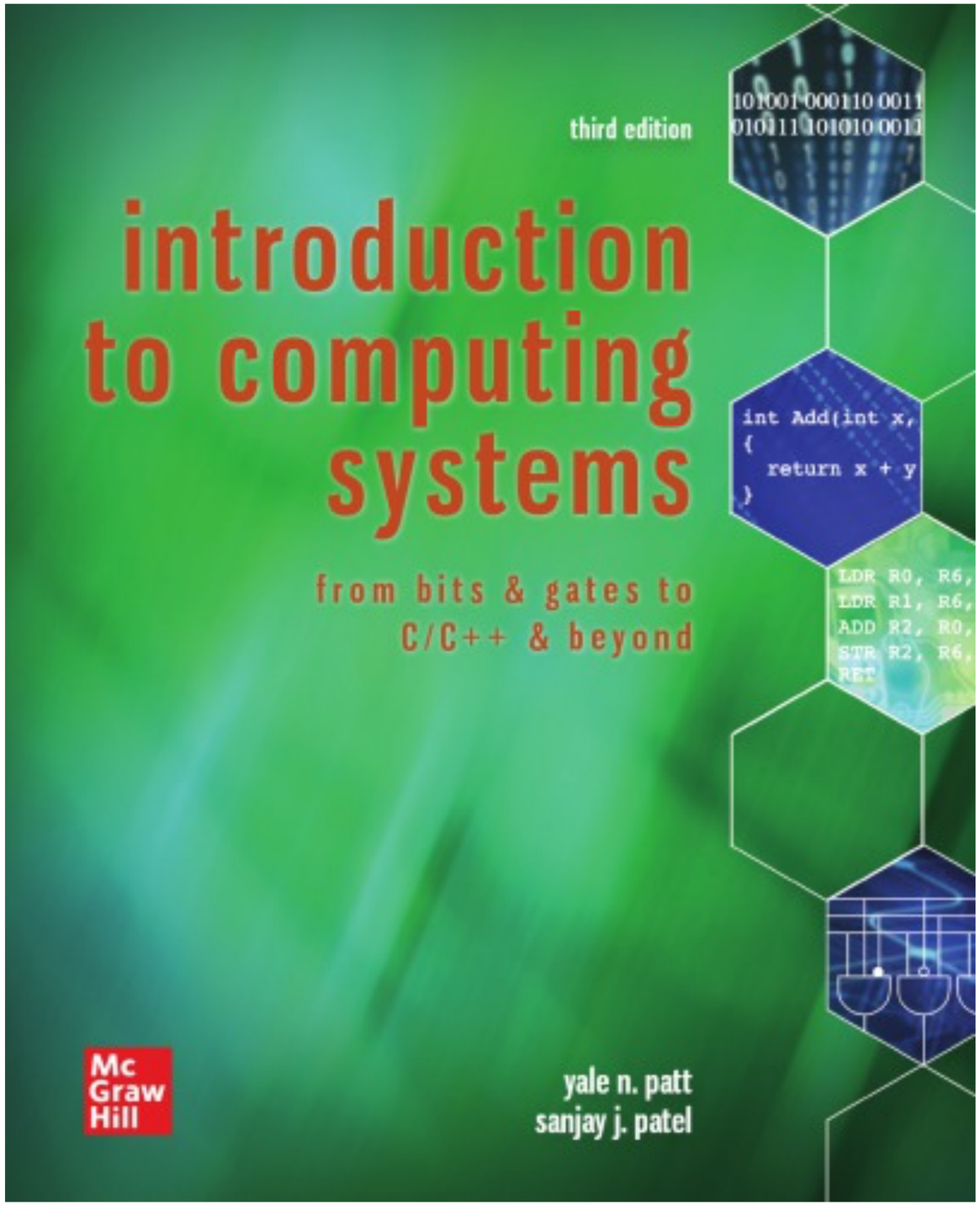
- Administrative Affairs
- Why CSCI 260: Programming, Analyzing, and Debugging with idea of the hardware, so to survive with constraints.
- Things expected after CSCI 260: Concepts + Applications.
- Goal of CSCI 260: Willingness of lifelong exposure to the hardware’s side.

Suggested Readings

Recommended Readings

Core Reading: Programming

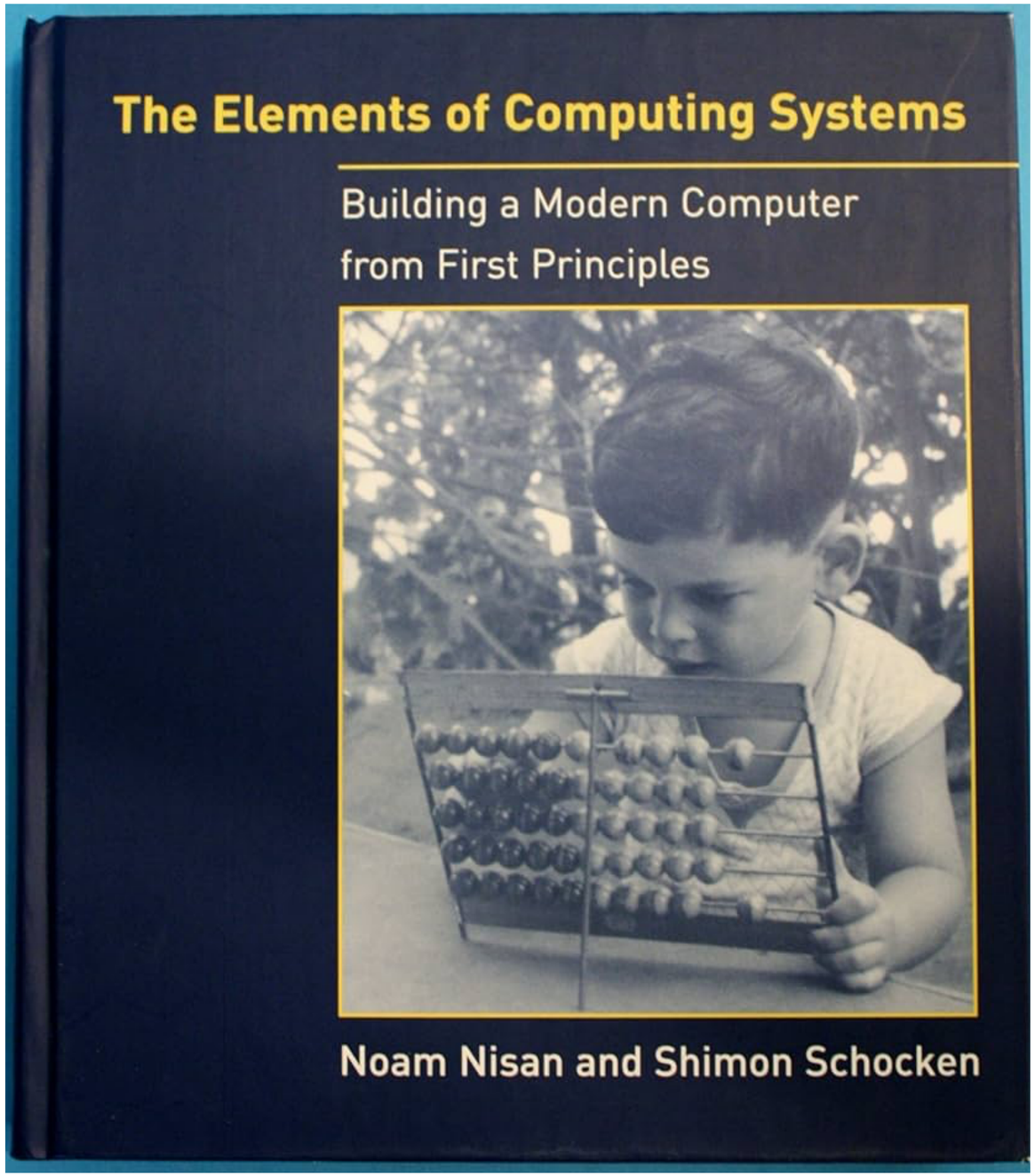
- Computer Organization
- Bit Representation
- LC-3 Assembly
- Introduction to C, via assembly.



Recommended Readings

Core Reading: Engineering

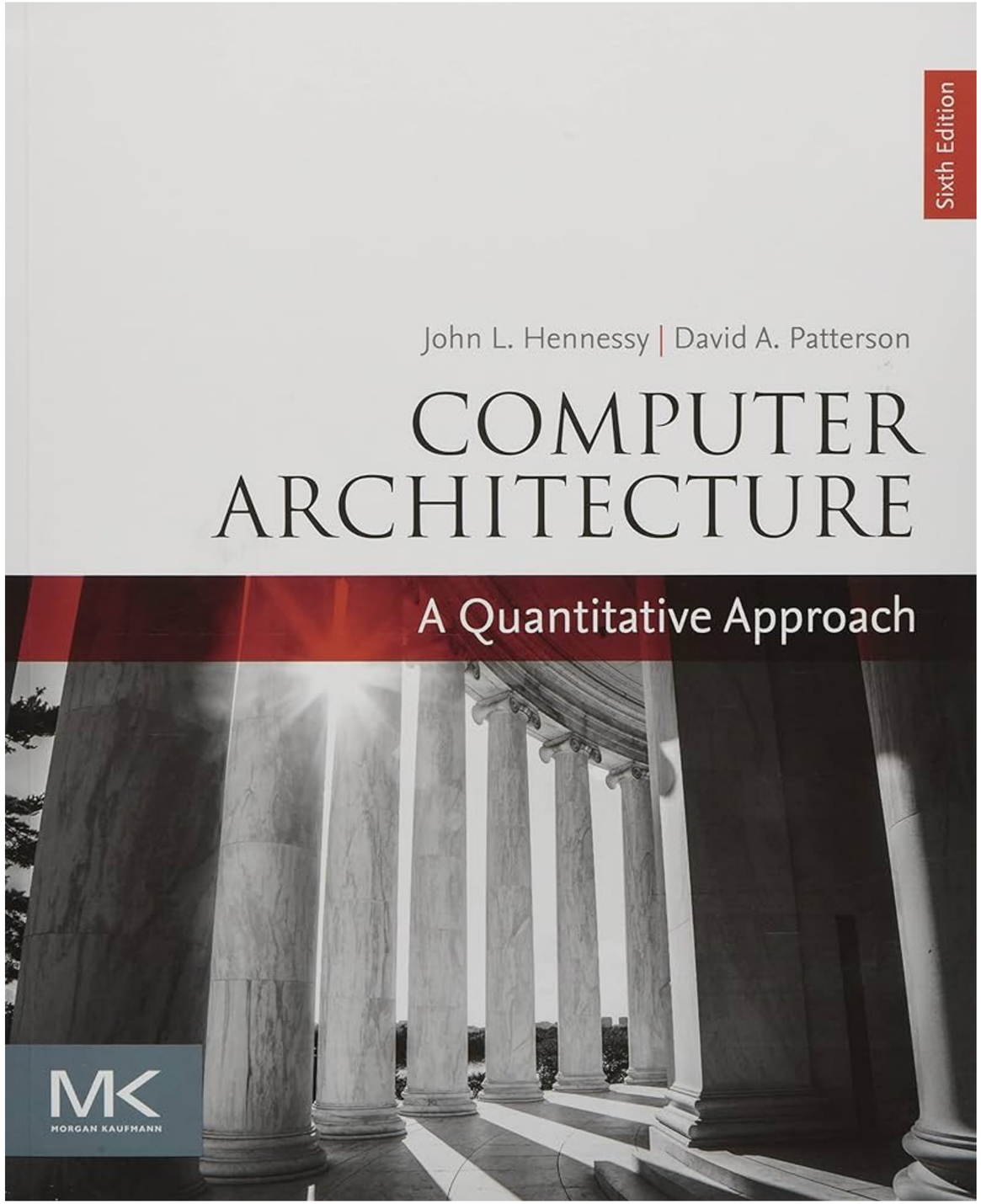
- Boolean Logic to OS
- Programming Language
- Assembler and Compiler



Recommended Readings

Advanced Reading

- Computer Design
- Performance Evaluation
- Principles of ISA Design



Brief History of Computers

Brief Timeline for Modern Computers

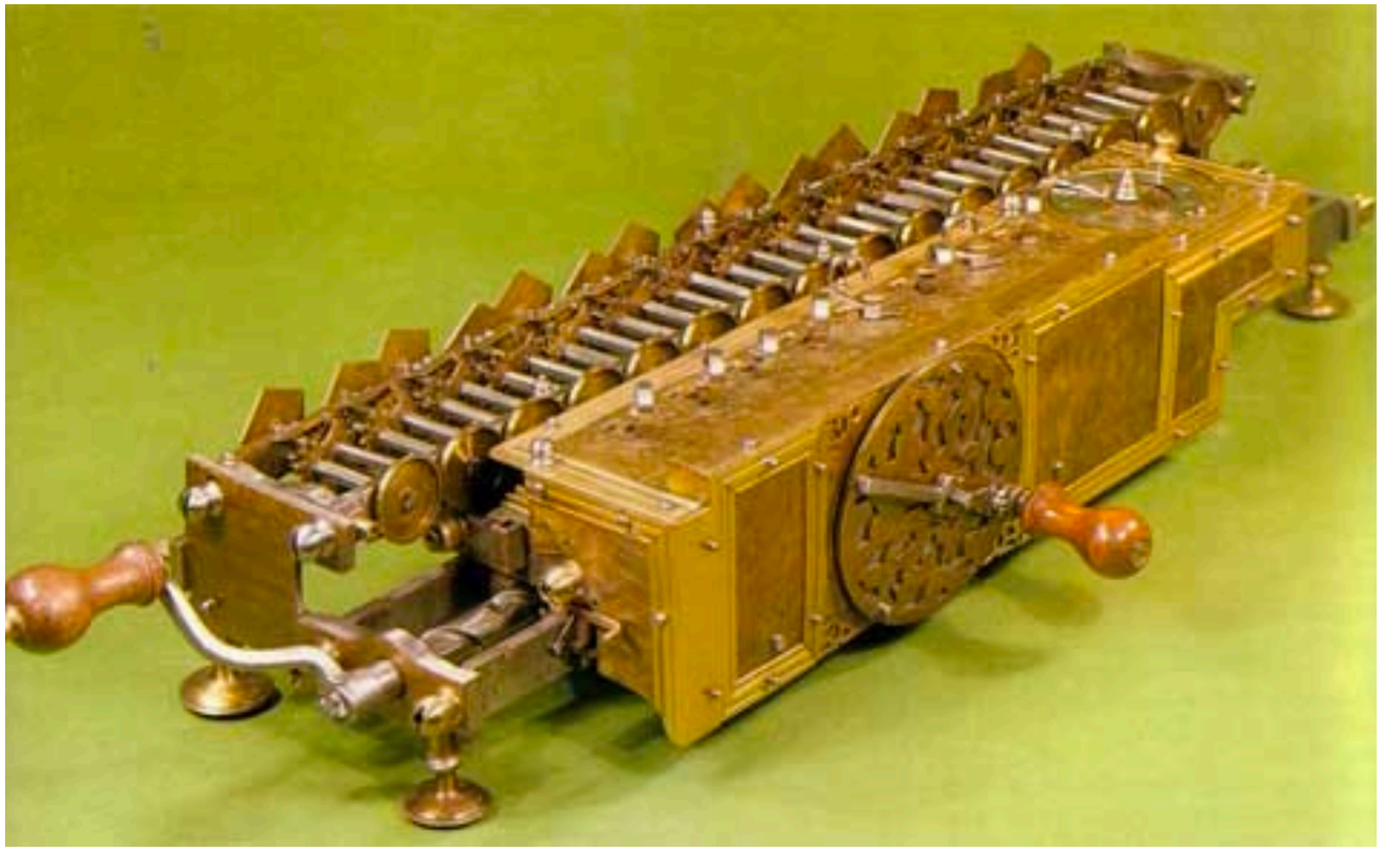
… is a way of evolutions of computation.

- Zeroth Generation -1945: Mechanical Computers
- First Generation - 1955: Vacuum Tubes
- Second Generation - 1965: Transistors
- Third Generation – 1980: Integrated Circuits
- Fourth Generation - : Very Large Scale Integration
- Fifth Generation: The ‘invisible’ computer

Mechanical Computers

“‘Automatic/Programmed’ Computation”

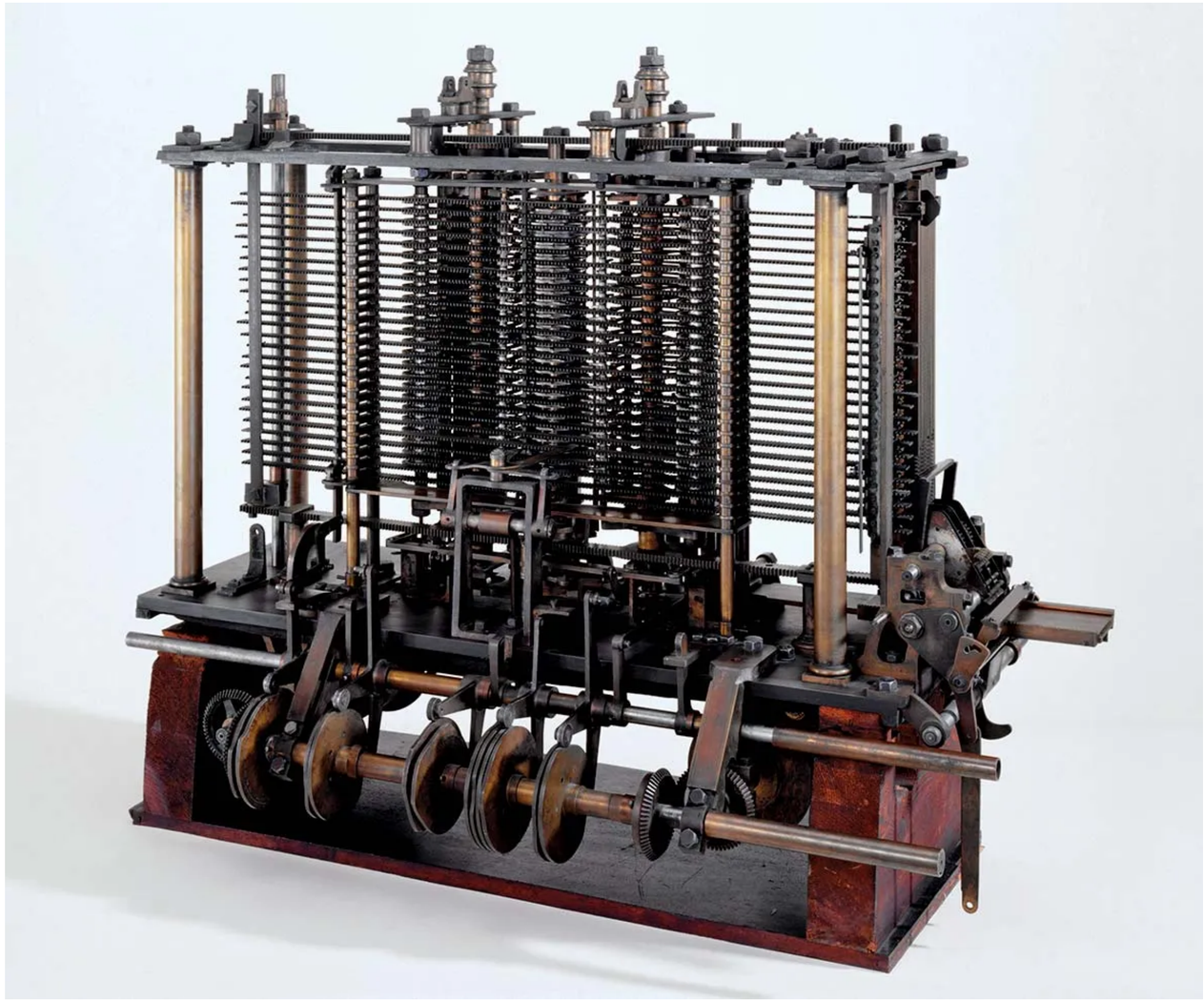
- Gottfried Leibniz, 1670: Mechanical machine capable of addition, subtraction, multiplication, and division.



Mechanical Computers

“‘Automatic/Programmed’ Computation”

- Charles Babbage, 1822: A difference engine is an automatic mechanical calculator designed to tabulate polynomial functions.
- The name difference engine is derived from the method of finite differences, a way to interpolate or tabulate functions by using a small set of polynomial coefficients.
- Not complete due to lack of precision in manufacturing.



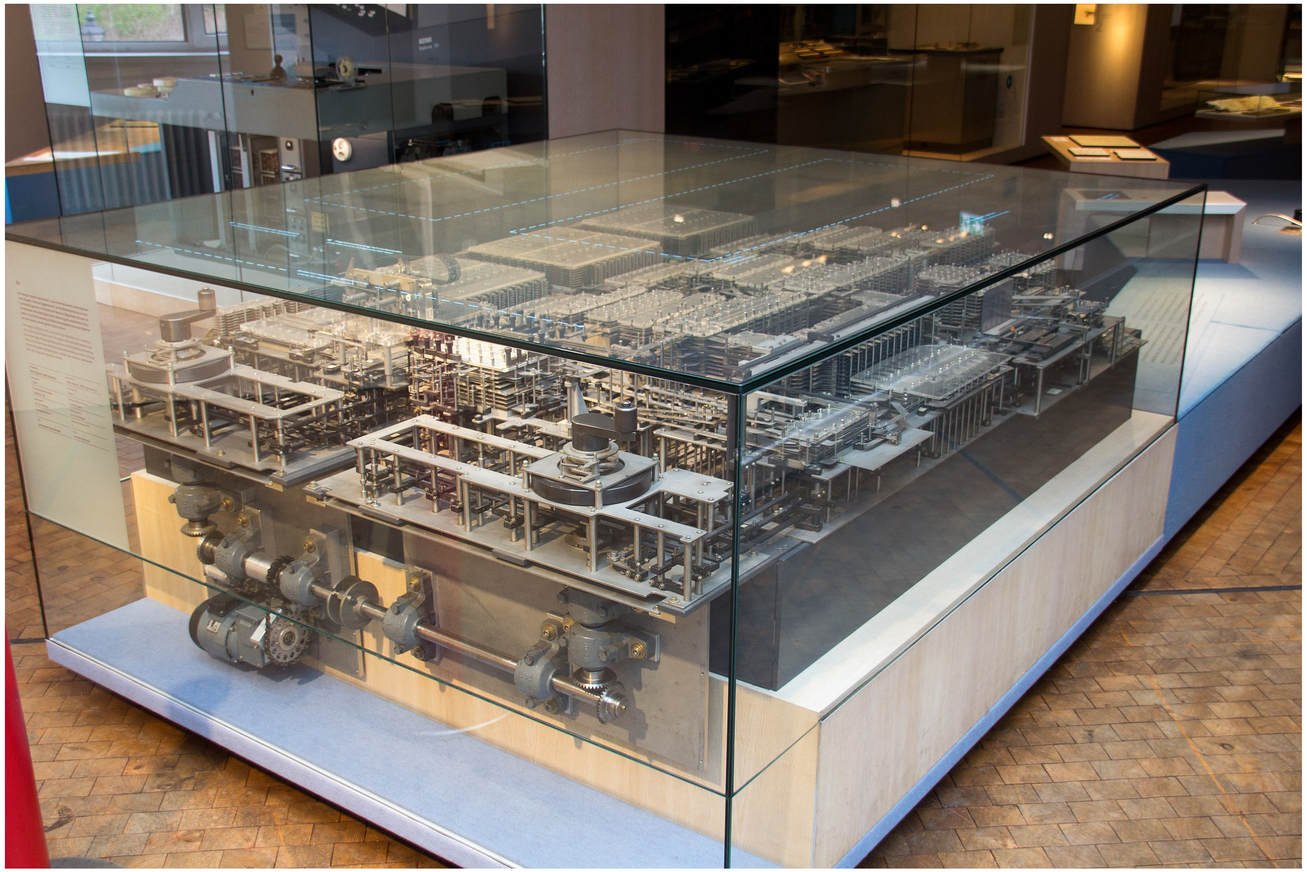
The Dawn of Computer Organization

- Question:
  - How to structure / organize computational machines?
  - How to control and steer execution?
- Original work (1946)
  - Burks, Goldstine, von Neumann: Preliminary discussion of the logical design of an electronic computing instrument.
- Result: von Neumann Architecture, the most dominant architecture – even today !

Mechanical Computers

“‘Automatic/Programmed’ Computation”

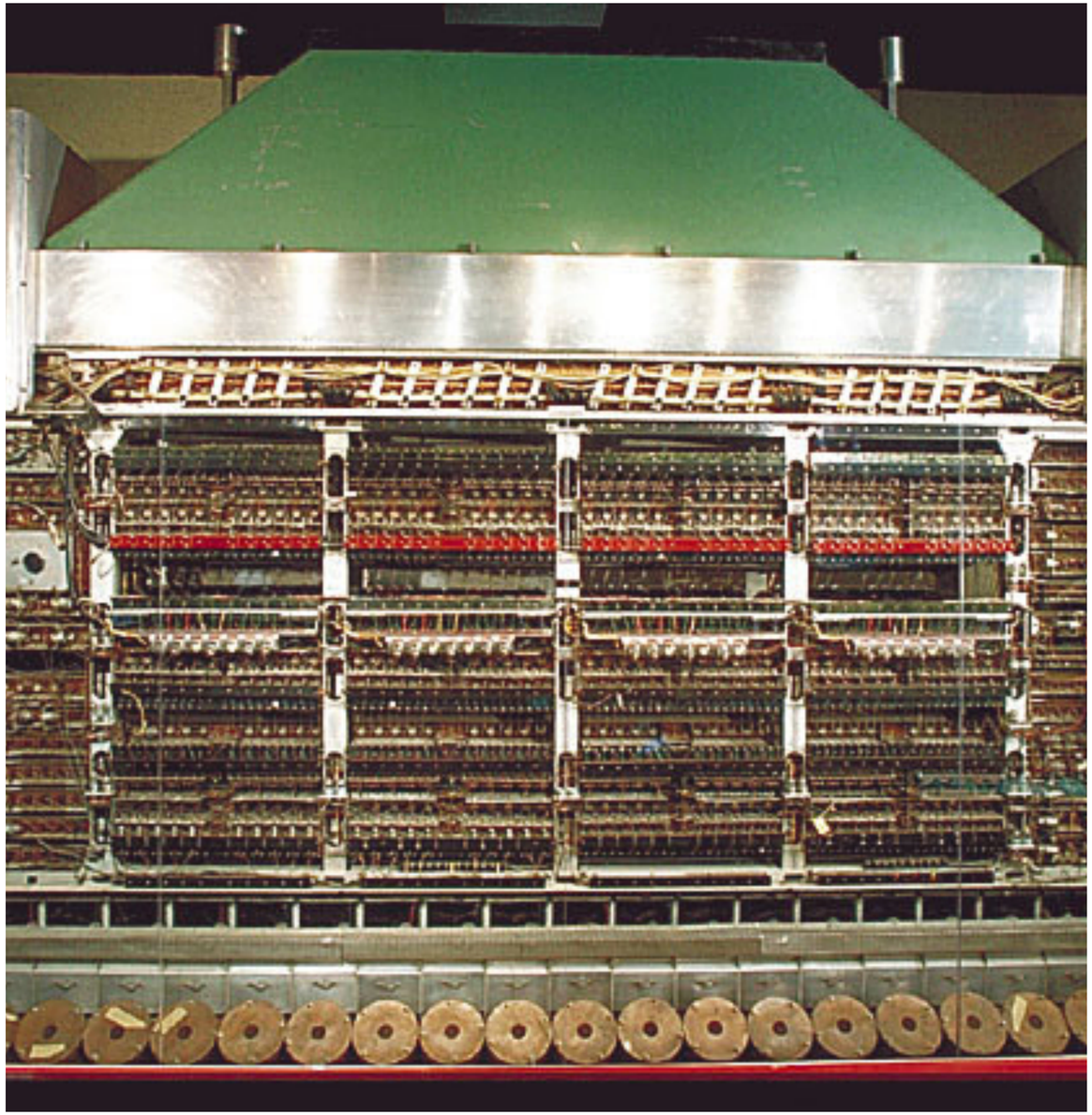
- Konrad Zuse, Z1, 1936
- First freely mechanical programmable computer in the world that used Boolean logic and binary floating-point numbers.
- However, it was unreliable in operation



Vacuum Tube Computers

“Computation with Electricity”

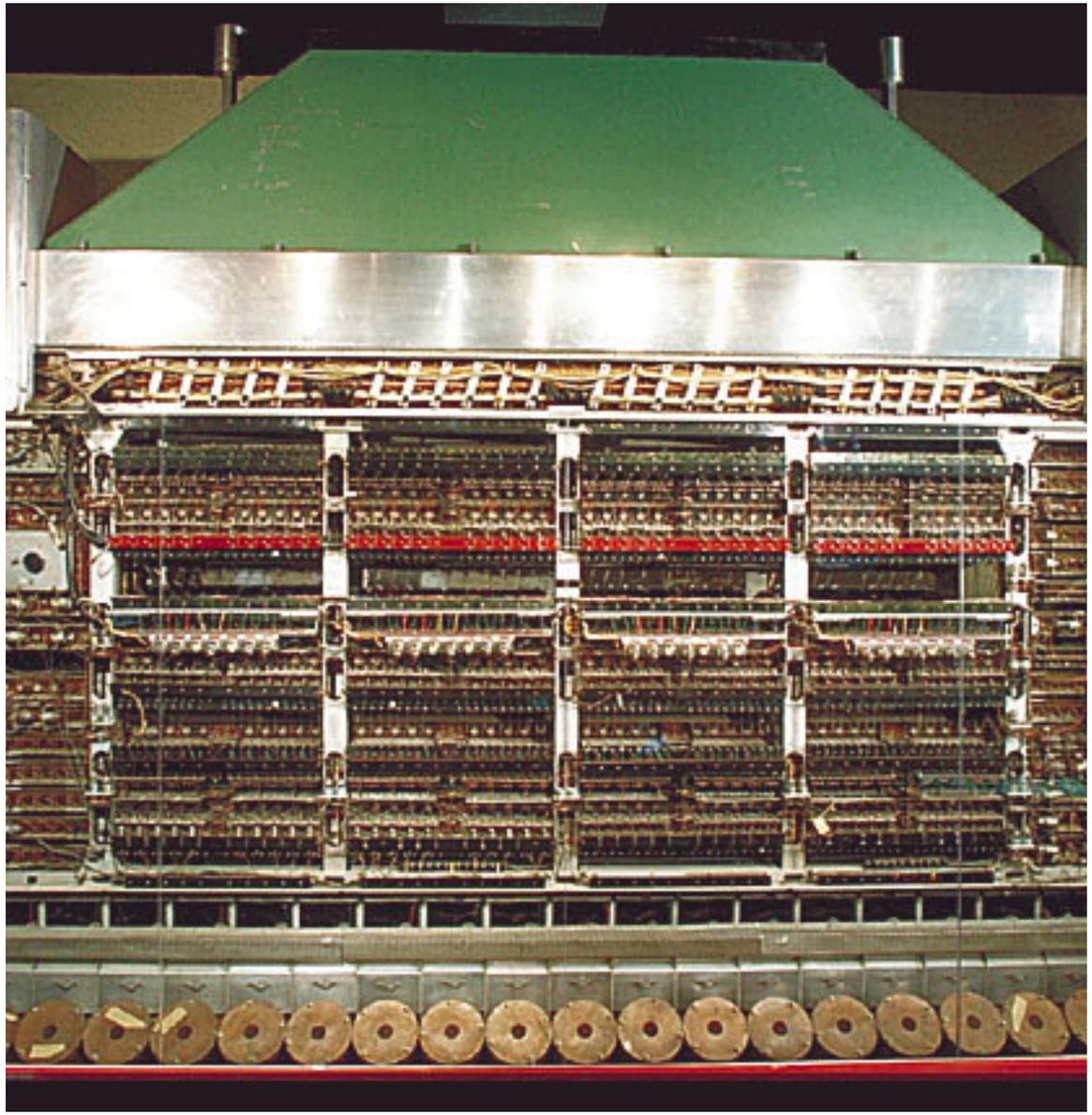
- Developed 1952 by von Neumann
- First machine based on his design principle
- Institute for Advanced Studies computer



Vacuum Tube Computers

“Computation with Electricity”

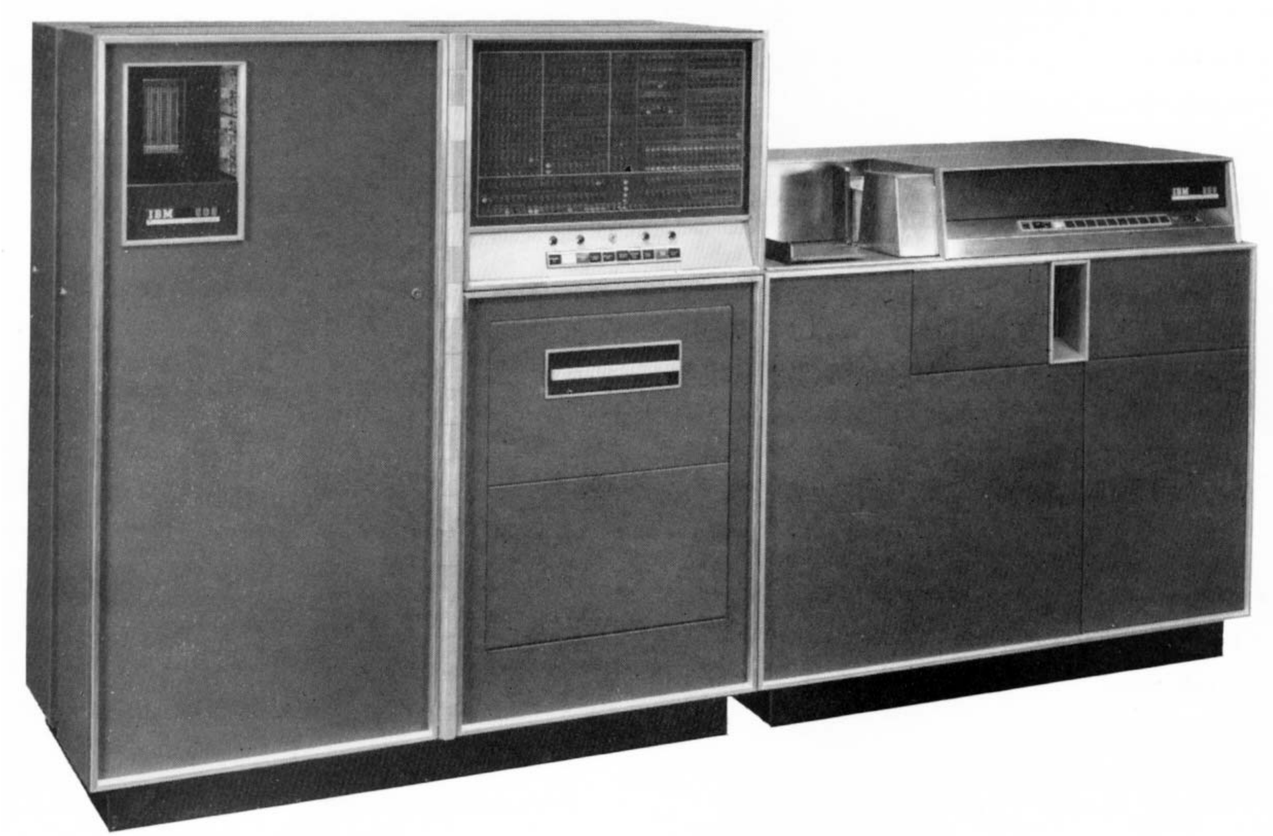
- General purpose machine
  - Independent of applications
  - Flexible & Programmable
- 4 main units
  - Control unit (Instruction counter)
  - Arithmetic unit (Accumulator)
  - Input/Output unit (Connection to the outside)
  - Main memory
- Interconnected by simple buses



Transistor Computers

“Efficient Computing”

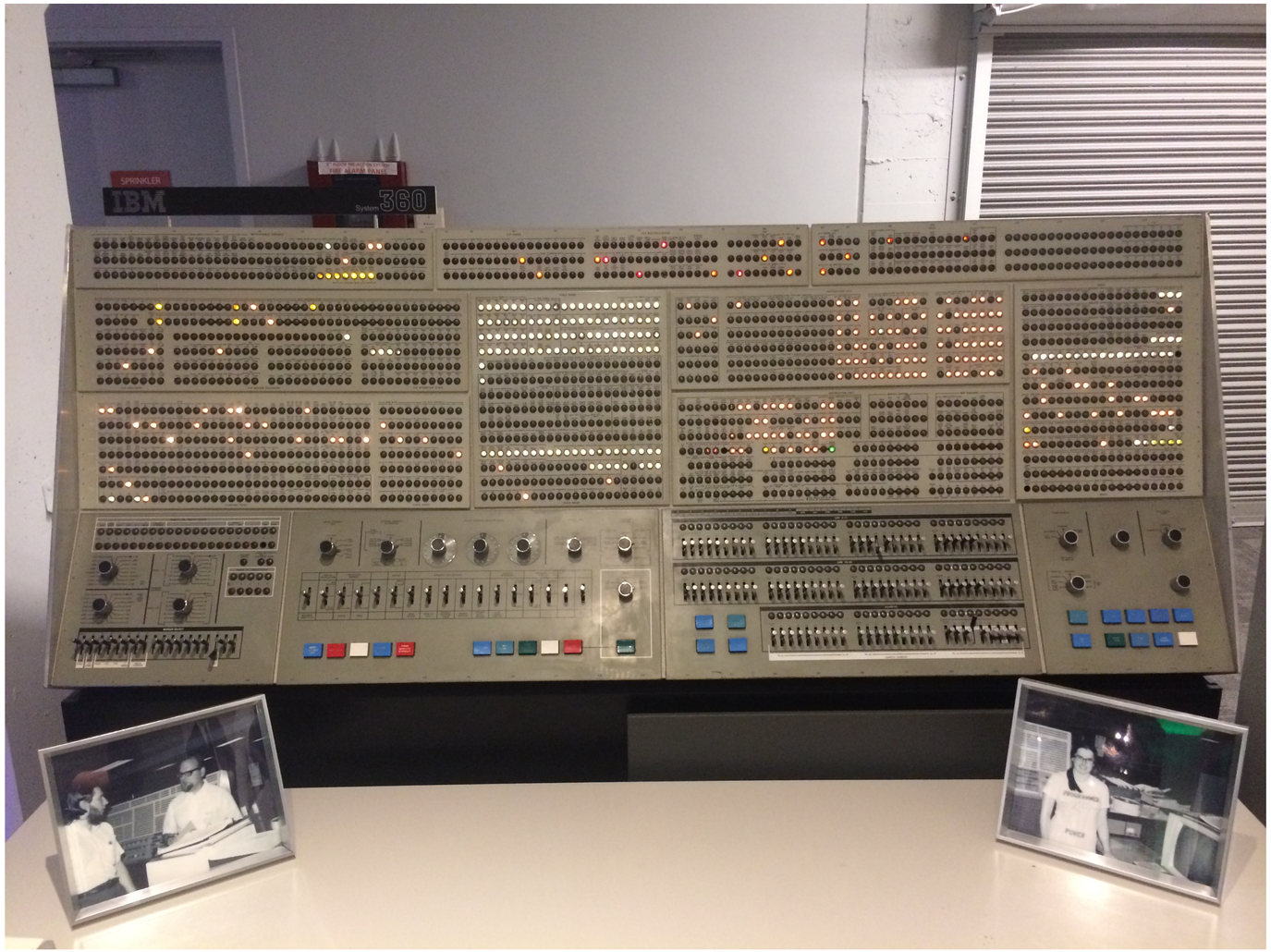
- IBM608(1955): The first IBM product to use transistor circuits without any vacuum tubes.
- Believed to be the world's first all-transistorized calculator to be manufactured for the commercial market.



Integrated Circuits

“Portable and Powerful”

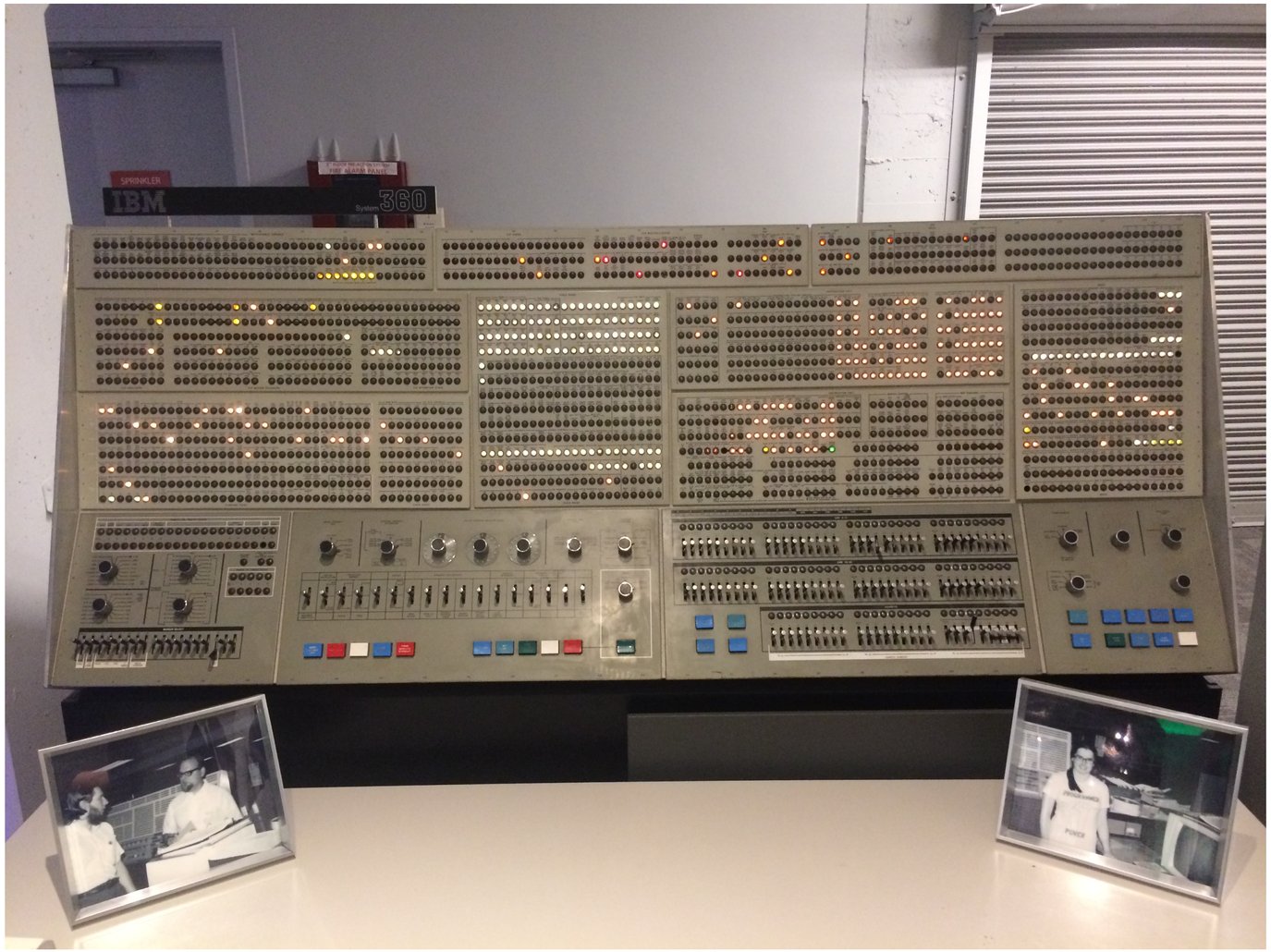
- IBM System 360(1964): The first family of computers designed to cover both commercial and scientific applications and a complete range of sizes from small, entry-level machines to large mainframes.
- The System/360 family introduced IBM's Solid Logic Technology (SLT), which packed more transistors onto a circuit card.



Integrated Circuits

“Portable and Powerful”

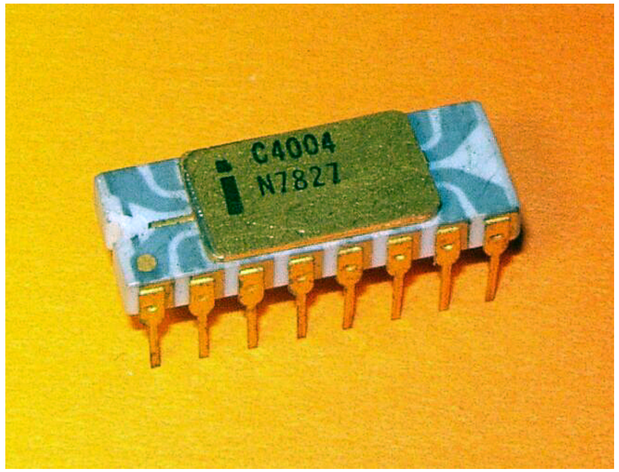
- IBM System 360(1964): The first family of computers designed to cover both commercial and scientific applications and a complete range of sizes from small, entry-level machines to large mainframes.
- The System/360 family introduced IBM's Solid Logic Technology (SLT), which packed more transistors onto a circuit card.

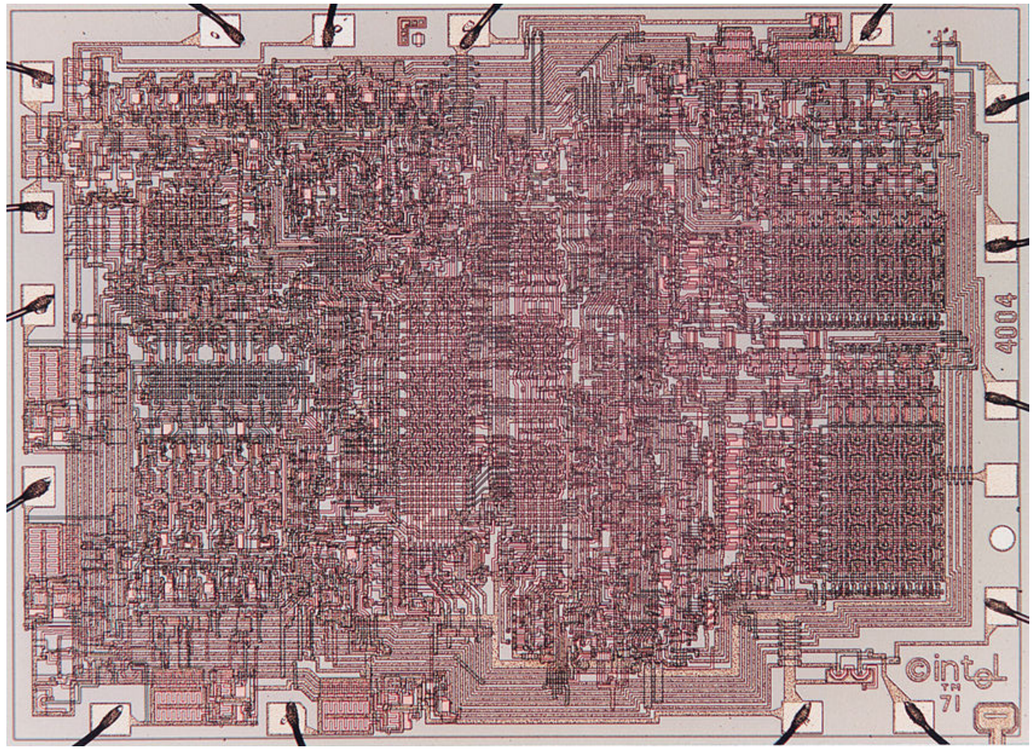


Very Large Scale Integration

“General Computing”

- Intel 4004(1971): The 4-bit 4004 CPU was the first significant commercial example of large-scale integration, showcasing the abilities of the MOS silicon gate technology (SGT).
- Compared to the existing technology, SGT enabled twice the transistor density and five times the operating speed, making future single-chip CPUs feasible.





‘Invisible Computer’

“Computing at another level”

- Nvidia DGX Data Center(2016): The Nvidia DGX (Deep GPU Xceleration) is a series of servers and workstations primarily geared towards enhancing deep learning applications through the use of general-purpose computing on graphics processing units (GPGPU).

What to consider when designing a computer?

What to consider when designing a computer?

As an expert in Computer Architecture

- Engineering discipline: explain, evaluate, compare, improve, and design …
  - … existing and future computing systems.
  - under investigation: systems, components, interconnect, ...
- Evaluation / optimisation criteria
  - desired functionality
  - non-functional properties (performance, dependability, …)
  - boundary conditions (technology, compatibility, …)

How to evaluate a computer’s performance?

Tech Trend: Rule of Thumb

… but that does not reflecting the whole picture of speed-up.

- Processor
  - Logic Capacity: ~30% increase per year
  - Clock Rate: ~20% increase per year
- Memory
  - DRAM Capacity: ~60% increase per year
  - Memory Speed: ~10% increase per year
  - Cost per Bit: ~25% decrease per year
- Disk
  - Capacity: ~60% increase per year

CPU Time

Reverse CPU Time = CPU Performance

$$\text{CPU time} = \text{Instruction Count} \times \text{CPI} \times \text{Clock Cycle Time}$$

$$\text{CPU time} = \frac{\text{Instruction Count} \times \text{CPI}}{\text{Clock Rate}}$$

- Instruction Count (IC)：Total instructions to execute
- CPI (Cycles Per Instruction)：How many clock cycles on average to execute an instruction
- Clock Cycle Time：How much time for a clock cycle（= 1 / Clock Rate）
- Clock Rate：CPU’s frequency（ 3.0 GHz）

CPU Time - Takeaways

Frequency(Clock Cycle Time) does not mean much!

$$\text{CPU time} = \text{Instruction Count} \times \text{CPI} \times \text{Clock Cycle Time}$$

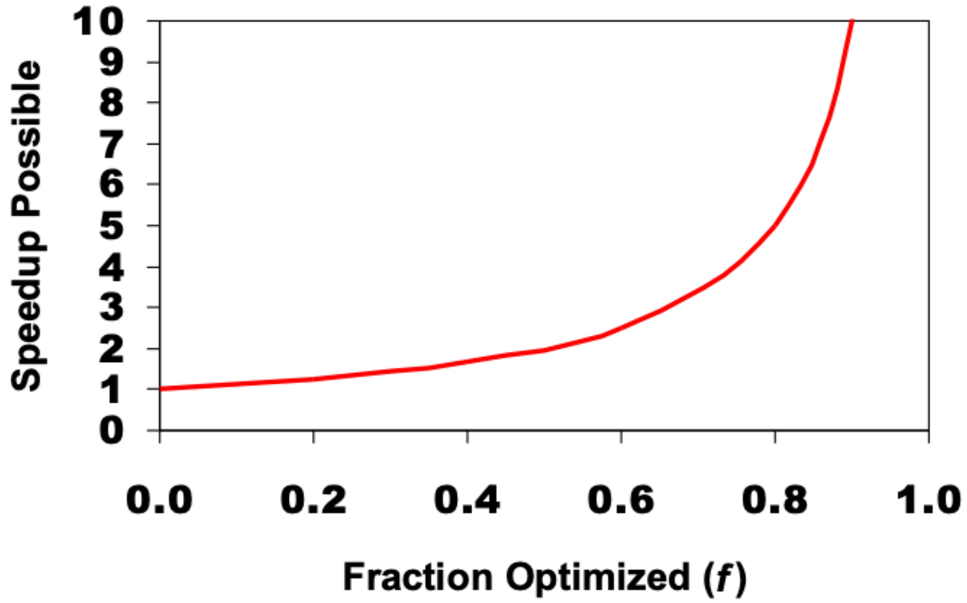
- Instruction Count —Program's Efficiency —Compilers and ISA
- CPI — ISA and Microarchitecture
- Microarchitecture and Technology

Amdahl's Law

Fraction optimized limits overall speedup

$$Speedup = \frac{1}{1 - f + \frac{f}{s}}$$

where f is fraction optimized,
s is speedup of that fraction

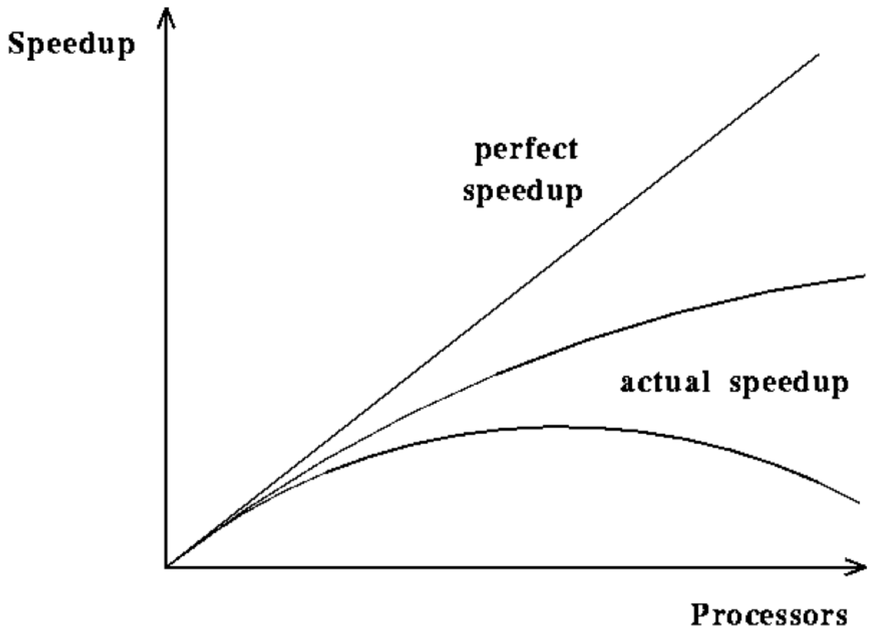


Parallelism, come with a price.

Plus, hindered by the work sequential.

- Parallel Processing - throw more processors at problem
  - 1024 parallel processors - LOTS OF MONEY!
  - 90% of code is parallel (f = 0.9)
  - Parallel portion speeds up by 1024 (s = 1024)
  - Serial portion of code (1-f) limits speedup

Takeaway: speed up does not come with more cores or threads. It works depending on the architecture, the compiler, and YOU.



Next Up:
More evaluations
Openers to bits and data in binaries.